Radar Tracker — Project Specification

Oct 11, 2026 · @Guido Asbun

Overview

Radar Tracker is a C++20 multi-target tracking system: a simulated radar produces noisy, cluttered detections of several aircraft, and a tracker recovers each aircraft's position and velocity with Kalman filters, scored against hidden ground truth.

It exists to show employers such as Northrop Grumman and Raytheon the skills their C++ postings ask for: estimation and sensor processing, real-time data flow over a network, disciplined modern C++ (tests, sanitizers, static analysis, CI), and containerized deployment. Every milestone ends with something runnable and a measured result, so the repository demonstrates competence instead of claiming it.

At a glance: a scenario file defines targets and sensor noise; `radar_sim` moves the targets and emits measurements; `radar_tracker` gates, associates, and filters them into tracks; `radar_score` compares tracks with truth and reports RMSE, continuity, and false-track count; `tools/plot.py` draws the result. From Milestone 3 the simulator and tracker run as separate processes over UDP under Docker Compose.

Proposed repository: `github.com/guidoasbun/radar-tracker`. Scope is v1 as defined below; extensions are listed at the end.

Goals and non-goals

The project commits to a tracker that is measurably better than the raw sensor, runs as two networked services, and is built to a standard a defense code reviewer would accept.

Goals (v1)

- Track 4 or more simultaneous targets through clutter and missed detections, with confirmed-track position RMSE at most half the raw measurement RMSE on straight-line targets.
- Keep one confirmed track per target for at least 95% of scans after confirmation, with zero false confirmed tracks in the baseline scenario.
- Run end to end with one command (`docker compose up`) and reproduce identically run to run through fixed random seeds.
- Pass CI on every push: build with `-Werror` on GCC and Clang, unit and integration tests, clang-tidy, clang-format, AddressSanitizer and UndefinedBehaviorSanitizer.
- Document the design (this spec, a README with architecture diagram and result plots) so a reviewer can judge the work without running it.

Non-goals (out of scope for v1)

- Real radar hardware or signal processing (RF, pulse compression, Doppler); the sensor is a statistical model.
- 3D tracking; targets move in a 2D horizontal plane.

- Polar (range and bearing) measurements and the Extended Kalman Filter; measurements are Cartesian. Deferred, see Open questions.
- Advanced association (JPDA, MHT) and maneuver models (IMM); listed under Extensions.
- Classification, identification, or threat assessment of targets.
- A production GUI; the optional live view (Milestone 4) is a demo aid.
- Native Windows (MSVC) builds; WSL2 is the supported path on Windows.

Terminology

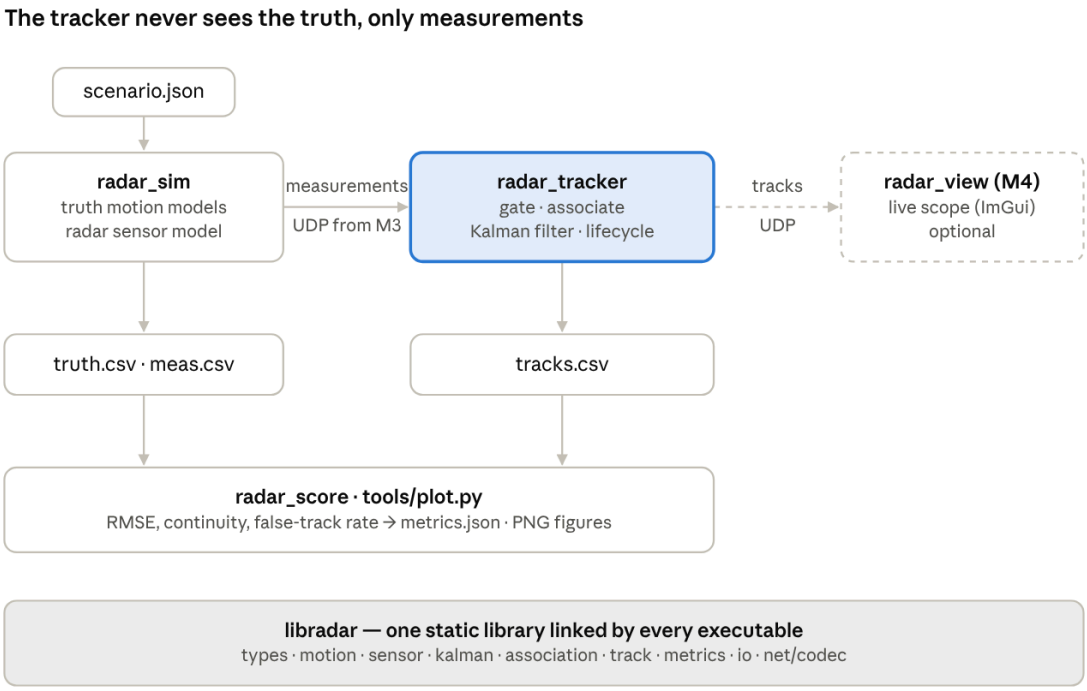
These terms are used with the meanings below throughout the spec and the code; the tracking literature uses several names for some of them, so synonyms are listed.

Term	Meaning in this project
Truth	The simulator's exact target states. Written to truth.csv for scoring only; the tracker never reads it.
Scan	One sensor sweep at interval $\Delta t = 0.1$ s. Each scan yields zero or more measurements.
Measurement (detection, plot)	A noisy 2D position (x, y) in meters reported by the sensor, from a target or from clutter.
Clutter (false alarm)	A measurement with no target behind it, placed uniformly over the surveillance area.
P_D	Probability that a target in view produces a measurement on a given scan (default 0.9).
Track	The tracker's estimate of one target: state vector, covariance, status, and hit/miss history.
State vector	[p_x, p_y, v_x, v_y]: position (m) and velocity (m/s) in the East–North frame.
Covariance (P)	4×4 matrix of the filter's uncertainty about the state; its position block draws the error ellipse.
Predict / update	The two Kalman steps per scan: propagate the state forward in time, then correct it with a measurement.
Innovation (residual)	Measurement minus predicted measurement; its covariance S sets the gate.
Gating	Rejecting measurements whose Mahalanobis distance from a track's prediction exceeds a χ^2 threshold.

Term	Meaning in this project
Association	Deciding which measurement, if any, belongs to each track on a scan (nearest neighbor here).
Tentative / confirmed / coasting	Track statuses: newly started; proven by M-of-N hits; confirmed but missed this scan, predicting only.
M-of-N logic	Confirm a tentative track after M hits in its first N scans (default 3 of 5).
RMSE	Root-mean-square position error against truth, in meters, over all scans of a confirmed track.
NIS	Normalized innovation squared; its average should be about 2 (the measurement dimension) when the filter is well tuned.
Continuity	Fraction of scans, after confirmation, on which a target has exactly one confirmed track.
False track	A confirmed track whose associated measurements come mostly from clutter.

System architecture

The simulator and the tracker are separate components that share one library and communicate only through measurements, so the tracker can never cheat by reading truth.



system architecture · executables, files, and the shared library

The highlighted tracker is the core deliverable; the dashed `radar_view` is optional and arrives last. Per-scan data flow:

1. `radar_sim` advances every target by Δt under its motion model, then asks the sensor model for this scan's measurements (detections with noise, plus clutter). Truth and measurements go to CSV.
2. Measurements reach `radar_tracker` as a `Scan` value: through a function call in Milestones 1 and 2, then as a UDP datagram decoded by `net/codec` from Milestone 3.
3. The tracker predicts every live track to the scan time, gates and associates the measurements, updates matched tracks, starts tentative tracks from the leftovers, and applies the lifecycle rules.
4. Confirmed and coasting tracks are appended to tracks.csv and, from Milestone 3, published as a TRACKS datagram for `radar_view` or a recorder.
5. After the run, `radar_score` pairs tracks with truth and writes metrics.json; `tools/plot.py` draws truth, measurements, tracks, and error ellipses.

The library and executable split keeps every piece of logic in `libradar`, where it is unit-tested; the executables under `apps/` only parse arguments, load config, wire the modules together, and loop.

Simulation model

The simulation is a 2D, discrete-time model: a 40 km × 40 km surveillance square centered on the radar, scanned every 0.1 s, with targets that fly straight lines and coordinated turns at aircraft speeds.

Frame, time, randomness

- Frame: East–North Cartesian, meters, origin at the radar. x is east, y is north. No altitude.
- Time: $t = k \cdot \Delta t$ with $\Delta t = 0.1$ s; default run 60 s (600 scans). Times are seconds as `double`.
- Randomness: one `std::mt19937_64` seeded from the scenario (default 42), so every run reproduces exactly.

Target motion models (truth)

Outside a maneuver a target flies constant velocity (CV): $p_{k+1} = p_k + v_k \cdot \Delta t$. During a maneuver segment it flies a coordinated turn (CT) at turn rate ω, applied as a rotation of the velocity followed by an Euler position step (the step error is negligible at $\Delta t = 0.1$ s):

$$\begin{bmatrix} v_x \\ v_y \end{bmatrix}_{k+1} = \begin{bmatrix} \cos\omega\Delta t & -\sin\omega\Delta t \\ \sin\omega\Delta t & \cos\omega\Delta t \end{bmatrix} \begin{bmatrix} v_x \\ v_y \end{bmatrix}_k, \qquad \mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_{k+1}\,\Delta t$$

A target may have several maneuver segments, each with its own time window and turn rate. Targets may enter or leave the surveillance square; outside it they produce no detections.

Sensor model (per scan)

1. For every target inside the square, draw a Bernoulli trial with probability P_D; on success emit $z = p + n$ with $n \sim N(0, \sigma_m^2 \cdot I)$.
2. Draw the clutter count from Poisson(λ) and place each clutter point uniformly in the square.
3. Shuffle the measurement list so its order carries no information about origin.
4. Record a hidden origin tag (target id, or -1 for clutter) for scoring only. The `Measurement` type handed to the tracker has no such field, so leakage is prevented by the type system, not by discipline.

Parameter	Symbol	Default	Unit	Notes
Scan interval	Δt	0.1	s	10 Hz
Run duration	T	60	s	600 scans
Surveillance half-width	L	20 000	m	square $\pm L$ on both axes
Measurement noise, per axis	σ_m	50	m	Gaussian, independent axes
Detection probability	P_D	0.9	—	per target, per scan
Clutter rate	λ	2	per scan	Poisson count, uniform placement
Target speed	$\lVert v \rVert$	100–300	m/s	set per target in the scenario

Parameter	Symbol	Default	Unit	Notes
Turn rate	ω	≤ 3	°/s	per maneuver segment; 3°/s is a standard-rate turn
RNG seed	—	42	—	one generator for the whole run

Scenario file

Scenarios are JSON files under `scenarios/`; the tracker block travels with the scenario so a run is fully described by one file.

```
{
  "dt_s": 0.1,
  "duration_s": 60,
  "seed": 42,
  "area_half_width_m": 20000,
  "sensor": { "sigma_m": 50.0, "p_detect": 0.9, "clutter_per_scan": 2.0 },
  "tracker": { "sigma_a": 5.0, "gate_chi2": 9.21, "confirm_m": 3, "confirm_n": 5,
"delete_after_misses": 5 },
  "targets": [
    { "id": 1, "x0": -15000, "y0": -12000, "vx0": 250, "vy0": 180, "maneuvers": [] },
    { "id": 2, "x0": 14000, "y0": 9000, "vx0": -200, "vy0": -120,
      "maneuvers": [ { "t_start_s": 20, "t_end_s": 35, "turn_rate_deg_s": 2.0 } ] }
  ]
}
```

Tracking algorithm

Each scan, the tracker predicts every track forward, admits only measurements inside each track's statistical gate, assigns them by nearest neighbor, updates with a linear Kalman filter, and applies M-of-N lifecycle rules.

Kalman filter (constant velocity, linear)

The state holds position and velocity; the transition matrix F advances position by velocity over Δt, and H selects the position because the sensor measures only position.

$$\mathbf{x} = \begin{bmatrix} p_x \\ p_y \\ v_x \\ v_y \end{bmatrix}, \qquad \mathbf{F} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Process noise Q follows the discrete white-noise-acceleration model with acceleration standard deviation σ_a; measurement noise R comes straight from the sensor's σ_m.

$$\mathbf{Q} = \sigma_a^2 \begin{bmatrix} \tfrac{\Delta t^4}{4} & 0 & \tfrac{\Delta t^3}{2} & 0 \\ 0 & \tfrac{\Delta t^4}{4} & 0 & \tfrac{\Delta t^3}{2} \\ \tfrac{\Delta t^3}{2} & 0 & \Delta t^2 & 0 \\ 0 & \tfrac{\Delta t^3}{2} & 0 & \Delta t^2 \end{bmatrix}, \qquad \mathbf{R} = \begin{bmatrix} \sigma_m^2 & 0 \\ 0 & \sigma_m^2 \end{bmatrix}$$

Predict, once per scan for every track:

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F}\,\hat{\mathbf{x}}_{k-1|k-1}, \qquad \mathbf{P}_{k|k-1} = \mathbf{F}\,\mathbf{P}_{k-1|k-1}\,\mathbf{F}^{\mathsf T} + \mathbf{Q}$$

Update, only for tracks that received a measurement z:

$$\mathbf{y} = \mathbf{z} - \mathbf{H}\,\hat{\mathbf{x}}_{k|k-1}, \qquad \mathbf{S} = \mathbf{H}\,\mathbf{P}_{k|k-1}\,\mathbf{H}^{\mathsf T} + \mathbf{R}, \qquad \mathbf{K} = \mathbf{P}_{k|k-1}\,\mathbf{H}^{\mathsf T}\,\mathbf{S}^{-1}$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}\,\mathbf{y}, \qquad \mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}\mathbf{H})\,\mathbf{P}_{k|k-1}\,(\mathbf{I} - \mathbf{K}\mathbf{H})^{\mathsf T} + \mathbf{K}\,\mathbf{R}\,\mathbf{K}^{\mathsf T}$$

The covariance update is the Joseph form: it keeps P symmetric and positive semi-definite under rounding, where the textbook $(I - KH)P$ does not. Store P as `Eigen::Matrix4d` and symmetrize after each update, $P = (P + P^T)/2$, as a cheap extra guard.

Track initialization. A tentative track starts from one unclaimed measurement: $\hat{x} = [z_x, z_y, 0, 0]^T$ and $P_0 = \mathrm{diag}(\sigma_m^2, \sigma_m^2, \sigma_{v0}^2, \sigma_{v0}^2)$ with $\sigma_{v0} = 150$ m/s, wide enough to cover any aircraft speed in the scenarios. Two-point differencing (velocity from the first two hits) is the documented upgrade if plots show slow convergence.

Gating and association (per scan)

1. Predict all tracks to the scan time.
2. For each track i and measurement j compute the innovation y_{ij} and the Mahalanobis distance $d^2_{ij} = y_{ij}^T S_i^{-1} y_{ij}$. Pairs with d^2 above the gate γ are not candidates.
3. Sort candidate pairs by d^2 ascending and walk the list, assigning a pair whenever neither its track nor its measurement is taken yet (greedy nearest neighbor).
4. Update each assigned track with its measurement and record a hit.
5. Tracks with no assignment record a miss and keep their prediction.
6. Measurements with no assignment start new tentative tracks.

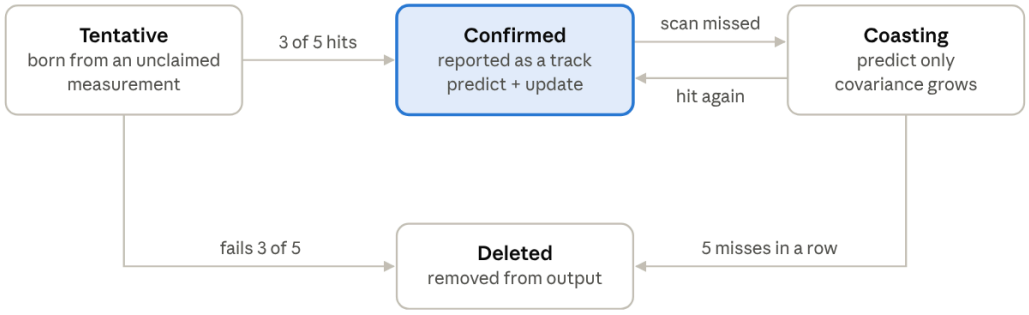
The gate is a χ^2 test on the 2D innovation:

$$d^2 = \mathbf{y}^{\mathsf T}\,\mathbf{S}^{-1}\,\mathbf{y} \le \gamma, \qquad \gamma = 9.21 \quad (\chi^2,\ 2\ \text{DOF},\ 99\%)$$

Stretch: replace step 3 with the Hungarian algorithm for a true minimum-total-cost assignment. The interface `Associator::assign(tracks, measurements)` returning a list of (track, measurement) pairs stays the same, so the swap is local.

Track lifecycle

A track earns confirmation with 3 hits in 5 scans and dies after 5 misses



track lifecycle · 4 states, 5 transitions

Only confirmed and coasting tracks appear in the output; a tentative track that fails 3-of-5 vanishes without ever being reported, which is what keeps clutter out of tracks.csv.

Parameter	Default	Meaning
confirm_m / confirm_n	3 / 5	Hits needed within a tentative track's first N scans to confirm it
delete_after_misses	5	Consecutive misses that delete a confirmed or coasting track (0.5 s at 10 Hz)
gate_chi2 (γ)	9.21	99% gate for a 2D innovation; 5.99 would be 95%
sigma_a	5 m/s²	Process-noise acceleration; raise for maneuvering targets, lower for smooth ones
sigma_v0	150 m/s	Initial velocity standard deviation for a new track

Tuning. The instrument is the mean NIS over a run ($\gamma \cdot S^{-1} \cdot \gamma$ averaged over updates). About 2 means σ_a and σ_m are balanced. Well above 2 means innovations are larger than the filter expects, so raise σ_a. Well below 2 means the filter is underconfident and smoothing too little, so lower σ_a. The maneuver scenario is where this trade-off shows.

Data formats and interfaces

All files are plain CSV with a header row and SI units; the UDP protocol is little-endian binary with a CRC-32 trailer, so corruption is detected and dropped rather than silently tracked.

CSV files

File	Written by	Columns	Notes
truth.csv	radar_sim	t_s, target_id, x_m, y_m, vx_mps, vy_mps	one row per target per scan while inside the square
meas.csv	radar_sim	t_s, meas_id, x_m, y_m, origin_id	origin_id is the target id or −1 for clutter; read only by radar_score and plot.py
tracks.csv	radar_tracker	t_s, track_id, status, x_m, y_m, vx_mps, vy_mps, pxx_m2, pyy_m2, pxy_m2	status is TENTATIVE, CONFIRMED, or COASTING; the P entries draw error ellipses
metrics.json	radar_score	see below	one object per run

```
t_s,track_id,status,x_m,y_m,vx_mps,vy_mps,pxx_m2,pyy_m2,pxy_m2
12.300,7,CONFIRMED,-3250.41,-9873.12,248.9,181.3,812.5,806.1,-14.2
```

metrics.json carries the acceptance numbers defined under Testing:

```
{
  "scenario": "baseline", "seed": 42, "scans": 600, "targets": 4,
  "raw_rmse_m": 70.9, "track_rmse_m": 31.4, "continuity": 0.98,
  "confirmed_tracks": 4, "false_tracks": 0, "mean_confirm_delay_s": 0.4, "mean_nis":
2.05
}
```

The numbers above are illustrative placeholders, not results.

UDP protocol (Milestone 3)

One datagram carries one scan, or one part of a scan. It is at most 1472 bytes, so it fits a standard Ethernet frame without IP fragmentation. Byte order is little-endian throughout; the codec uses `std::memcpy` into fixed-width integers, never a cast of a packed struct.

Offset	Size	Field	Type	Value / notes
0	4	magic	uint32	0x52444152 ("RADR")
4	2	version	uint16	1
6	2	msg_type	uint16	1 = MEASUREMENTS, 2 = TRACKS
8	4	seq	uint32	increments per datagram; gaps

Offset	Size	Field	Type	Value / notes
				are counted and logged
12	8	t_s	float64	scan time
20	1	part	uint8	0-based index when a scan spans several datagrams
21	1	parts	uint8	total datagrams for this scan
22	2	n_items	uint16	items in this datagram
24	n × item	items	—	MEASUREMENTS item (16 B): x_m, y_m as float64. TRACKS item (56 B): track_id uint32, status uint8, 3 pad bytes, then x_m, y_m, vx_mps, vy_mps, pxx_m2, pyy_m2 as float64
end − 4	4	crc32	uint32	CRC-32 (IEEE 802.3) over header and items

Limits follow from the size: at most 90 measurements or 25 tracks per datagram; a larger scan is split into `parts` datagrams sharing `t_s`. The decoder rejects a datagram on bad magic, unknown version, length mismatch, or CRC failure, counts the rejection, and continues. A scan is delivered to the tracker when all its parts have arrived, or when a datagram for a later scan arrives first (delivered partial, counted as such).

Default endpoints: radar_sim sends to 127.0.0.1:5005, where radar_tracker listens; radar_tracker publishes TRACKS to 127.0.0.1:5006 for radar_view or a recorder. Both are flags; under Docker Compose the service name replaces the address (`tracker:5005`).

Command-line interface

Command	Purpose	Example
radar_sim	run a scenario; write CSVs and/or stream UDP	`radar_sim --config scenarios/baseline.json --out out/ --track` (M1–M2, tracker in-process); `radar_sim --config … --udp 127.0.0.1:5005 --realtime` (M3)
radar_tracker	consume measurements; write tracks.csv and/or publish UDP	`radar_tracker --listen 0.0.0.0:5005 --publish 127.0.0.1:5006 --config scenarios/baseline.json --out out/`
radar_score	compare tracks with truth	`radar_score --truth out/truth.csv --meas out/meas.csv --tracks out/tracks.csv --json out/metrics.json`
radar_view	live display of tracks (M4)	`radar_view --listen 0.0.0.0:5006`
tools/plot.py	static figures	`python tools/plot.py out/ --save out/plot.png`

Exit codes: 0 success, 1 bad arguments or config (the message names the JSON path), 2 I/O or network error. `--realtime` paces datagrams at Δt; without it the simulator runs as fast as it can.

Repository layout and module responsibilities

All logic lives in one static library, `libradar`, with one header per module under `include/radar/`; the executables under `apps/` only parse arguments, load config, and wire modules together.

```
radar-tracker/
├── CMakeLists.txt              # project, options, add_subdirectory
├── CMakePresets.json           # debug, release, asan, ci
├── cmake/Dependencies.cmake    # FetchContent: Eigen, GoogleTest, nlohmann/json,
CLI11
├── .clang-format  .clang-tidy  .gitignore  LICENSE  README.md
├── include/radar/
│   ├── types.hpp               # Measurement, Scan, TrackState, TrackStatus
│   ├── motion.hpp              # CV and CT truth models
│   ├── sensor.hpp              # detection, noise, clutter
│   ├── kalman.hpp              # KalmanCV: predict, update, gate distance
│   ├── association.hpp         # Associator (greedy nearest neighbor; Hungarian
later)
│   ├── track.hpp               # Track: filter + status + hit/miss counters
│   ├── tracker.hpp             # Tracker: per-scan orchestration
│   ├── metrics.hpp             # RMSE, continuity, false tracks, NIS
│   ├── io/config.hpp           # scenario JSON -> structs
```

```
│   ├── io/csv.hpp              # CSV writers and readers
│   └── net/codec.hpp  net/udp.hpp   # M3: framing, CRC-32, sockets, reassembly
├── src/                        # one .cpp per header -> libradar
├── apps/
│   ├── radar_sim/main.cpp
│   ├── radar_tracker/main.cpp  # M3
│   ├── radar_score/main.cpp
│   └── radar_view/main.cpp     # M4, built only with -DRADAR_BUILD_VIEW=ON
├── tests/                      # GoogleTest: one file per module, plus integration/
├── scenarios/                  # single, baseline, crossing, maneuver,
dense_clutter (.json)
├── tools/plot.py  tools/requirements.txt
├── docker/Dockerfile  docker-compose.yml
└── .github/workflows/ci.yml
```

Module	Owns	Depends on	Milestone
types	plain structs shared everywhere; no logic	Eigen	M1
motion	truth propagation (CV, CT)	types	M1
sensor	P_D trials, Gaussian noise, Poisson clutter, shuffling, hidden origin tags	types, motion	M1 noise; M2 P_D and clutter
kalman	predict, Joseph-form update, innovation, S, d^2	types	M1
association	gated greedy nearest-neighbor assignment	kalman	M2
track	one track's filter, status, counters, lifecycle transitions	kalman	M2
tracker	per-scan loop over tracks and measurements; creation and deletion	association, track	M2
metrics	track-to-truth pairing by dominant origin; RMSE, continuity, false tracks, NIS	types	M1 RMSE; M2 the rest

Module	Owns	Depends on	Milestone
io	config parsing, CSV read and write	nlohmann/json	M1
net	datagram codec, CRC-32, UDP send and receive, scan reassembly	types	M3
apps/*	argument parsing and wiring only	libradar, CLI11	each

Dependency rule: arrows point one way. No module from `tracker` downward includes `io` or `net`, so the algorithms compile and test without any file or socket code. Headers under `include/radar/` are the public API; internal helpers stay in `src/`.

Technology stack and build

The toolchain is deliberately conventional: C++20, CMake with FetchContent so a fresh clone builds with no manual installs, Eigen for the linear algebra, GoogleTest for tests, and Docker for the two-service run.

Component	Choice	Minimum	Why
Language	C++20	GCC 12, Clang 15, Apple Clang 15	concepts, `std::span`, designated initializers; supported by every current toolchain
Build	CMake + Ninja	CMake 3.24	presets; FetchContent pins each dependency to a Git tag
Linear algebra	Eigen	3.4	header-only; fixed-size matrices allocate nothing on the heap; common in aerospace code
Tests	GoogleTest	1.14	de-facto standard; CTest integration
Config	nlohmann/json	3.11	header-only; readable scenario files with precise error paths
CLI	CLI11	2.3	header-only argument parsing with validation

Component	Choice	Minimum	Why
Plotting	Python 3.10, matplotlib, pandas	—	fast iteration on figures; not part of the C++ build
Live view (M4)	Dear ImGui, ImPlot, GLFW, OpenGL 3	—	immediate-mode GUI, the usual choice for engineering tools
Containers	Docker, Compose v2	—	two services on one network; multi-stage image under 100 MB
CI	GitHub Actions, ubuntu-24.04	—	matrix over gcc and clang; separate jobs for asan, tidy, format, docker

Build and run

```
cmake --preset release && cmake --build --preset release
ctest --preset release
./build/release/apps/radar_sim/radar_sim --config scenarios/single.json --out out/
--track
python tools/plot.py out/
```

Presets: `debug` (-O0 -g), `release` (-O2 -DNDEBUG), `asan` (debug plus AddressSanitizer and UndefinedBehaviorSanitizer), `ci` (release plus `-Werror` and clang-tidy). Dependencies are fetched by tag into `build/_deps` and cached there; `-DRADAR_USE_SYSTEM_DEPS=ON` switches to `find_package` for Docker layer caching.

Platforms. Linux (Ubuntu 22.04 or 24.04) is primary and what CI runs. macOS 13 or later builds with Apple Clang. Windows is supported through WSL2 only.

Docker (M3). One multi-stage `Dockerfile` builds all executables in a builder stage and copies them into a slim runtime image. `docker-compose.yml` defines two services, `sim` and `tracker`, on one network, with `out/` mounted as a volume so metrics.json and tracks.csv land on the host.

Code quality and engineering standards

Every push must compile warning-free under two compilers, pass the tests under sanitizers, and satisfy clang-tidy, because defense-sector reviewers read the CI configuration before the code.

Compiler and tooling

- Warnings: `-Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wsign-conversion -Wnon-virtual-dtor -Wold-style-cast`, promoted to errors with `-Werror` in the `ci` preset.
- Static analysis: clang-tidy with `bugprone-*`, `cppcoreguidelines-*`, `modernize-*`, `performance-*`, `readability-*`, `misc-*`; the few disabled checks are listed in `.clang-tidy` with a reason each.

- Formatting: clang-format, Google base style, 100 columns; CI runs `clang-format --dry-run --Werror`.
- Dynamic analysis: the `asan` preset runs the whole test suite under AddressSanitizer and UndefinedBehaviorSanitizer on every push; Valgrind memcheck runs nightly (it cannot share a build with ASan); ThreadSanitizer covers `net` and `tracker` tests from Milestone 3.
- Coverage: gcov and lcov report uploaded as a CI artifact; target 85% line coverage on `libradar`.

Safety-critical coding conventions

The README describes the code as aligned with JSF++ and MISRA C++ practice, never as compliant, and lists exactly these rules:

- No raw `new` or `delete`; ownership through values, `std::unique_ptr`, and standard containers.
- No heap allocation in the per-scan path after initialization: fixed-size `Eigen::Matrix<double, 4, 4>`, `std::vector`s reserved at startup, `std::span` views.
- No exceptions inside `libradar` algorithms; errors return `std::optional` or an error enum. Exceptions are allowed only at the app boundary, in config parsing and I/O.
- No recursion and no unbounded loops; every loop bound is a container size or a config constant.
- Fixed-width integers (`std::uint32_t`) and explicit endianness in `net/codec`; `std::memcpy`, never a `reinterpret_cast` of a packed struct.
- `const` by default; no implicit narrowing (`-Wconversion` enforces it).
- Every public function has a one-line Doxygen comment that states its units.

Git workflow

`main` is protected; work lands through pull requests with CI green, one per milestone or smaller. Commits follow Conventional Commits (`feat(kalman): add Joseph-form update`). Each milestone is tagged `v0.1` through `v0.4`.

CI pipeline (`.github/workflows/ci.yml`, on push and pull request)

1. Check out; install ninja, clang, clang-tidy, clang-format, lcov, valgrind.
2. Format check over `include`, `src`, `apps`, `tests`.
3. Build matrix (gcc, clang) with the `ci` preset; cache `build/_deps`.
4. `ctest --output-on-failure` (unit and integration tests).
5. Sanitizer job: `asan` preset, run ctest.
6. clang-tidy job over every translation unit; any warning fails the job.
7. Coverage job: gcov, lcov, upload the HTML report as an artifact.
8. Smoke run: `radar_sim` on `scenarios/baseline.json`, then `radar_score`; assert the thresholds from Testing; upload `plot.png` as an artifact.
9. From M3: build the Docker image; `docker compose up --abort-on-container-exit`; check that `metrics.json` was produced.
10. Nightly schedule: Valgrind memcheck on the test binaries.

Testing and verification plan

Correctness is proven at three levels: unit tests on each module, an end-to-end acceptance run whose metrics must clear fixed thresholds in CI, and a set of scenarios each designed to break one specific part.

Unit tests (GoogleTest, one file per module)

Test file	Checks
test_kalman.cpp	predict equals F·x analytically; noise-free measurements converge to the true velocity within 1 m/s in 50 steps; P stays symmetric and positive definite; Joseph update equals the textbook update to 1e-9
test_motion.cpp	CV path length equals speed × time; CT at 3°/s returns to its heading after 120 s; targets outside the square are flagged
test_sensor.cpp	with a fixed seed, noise mean ≈ 0 and standard deviation ≈ σ_m within 5% over 10^5 draws; P_D = 0 yields no detections; λ = 0 yields no clutter; measurement order is shuffled
test_association.cpp	the gate rejects $d^2 > \gamma$; greedy assignment never assigns a track or measurement twice; two well-separated tracks keep their identities
test_track.cpp	tentative → confirmed after 3 of 5 hits; deleted after failing 3 of 5; coasting → deleted after 5 misses; coasting → confirmed on a hit
test_tracker.cpp	one target with P_D = 1 yields exactly one confirmed track; clutter-only input yields zero confirmed tracks over 600 scans
test_metrics.cpp	RMSE of a hand-built track equals its closed-form value; continuity and false-track counts on a synthetic case
test_codec.cpp (M3)	encode → decode round-trips every field; a flipped bit fails the CRC; a truncated datagram is rejected; 91 measurements split into 2 parts and reassemble
test_config.cpp	every shipped scenario parses; a missing field reports its JSON path; invalid values (P_D > 1) are rejected

Acceptance metrics (radar_score on `scenarios/baseline.json`, seed 42; asserted by `tests/integration/test_acceptance.cpp` on every push)

Metric	Definition	Threshold
Raw measurement RMSE	RMS of $\|z - p_{true}\|$ over target-origin measurements	reported; about 71 m at σ_m = 50 m
Track position RMSE	RMS of $\|\hat{x}_{pos} - p_{true}\|$ over confirmed-track scans, each track paired with the truth target that produced most of its measurements	at most 0.5 × raw RMSE
Continuity	scans with exactly one confirmed track per target, divided by scans after that target's first confirmation	at least 0.95
False confirmed tracks	confirmed tracks whose dominant origin is clutter	0 on baseline; at most 1 on dense_clutter
Confirmation delay	time from a target's first detection to its confirmation, averaged over targets	at most 1.0 s
Mean NIS	average $y \cdot S^{-1} \cdot y$ over confirmed-track updates	between 1.5 and 2.6 (ideal 2.0)

Scenarios

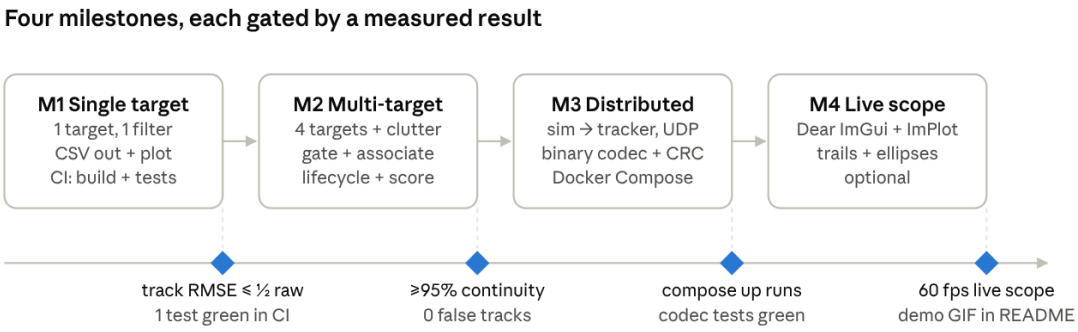
Scenario	Targets	P_D	λ (clutter per scan)	Designed to stress
single.json	1 CV	1.0	0	Milestone 1 filter correctness
baseline.json	4 CV, well separated	0.9	2	the acceptance thresholds above
crossing.json	2 CV whose paths cross near t = 30 s	0.9	2	association; track-swap detection
maneuver.json	2 targets, one in a 2°/s turn for 15 s	0.9	2	process-noise tuning (σ_a), lag during turns
dense_clutter.json	4 CV	0.8	20	false-track suppression, gate size

Determinism. Runs use one fixed seed and a single thread on the M1–M2 path, so results are repeatable on a given platform. Cross-compiler floating-point differences are absorbed by the threshold margins above, not by comparing output bytes.

Manual checks. The `plot.py` figures: tracks should hug truth, error ellipses should shrink after confirmation, and no track should follow clutter. For M3, `tracks.csv` from the UDP path must match the in-process path on the same seed, apart from scans lost to dropped datagrams (expected zero on localhost).

Milestones and acceptance criteria

Four milestones, each ending in something runnable and a number that proves it; Milestone 4 is optional polish and is gated behind Milestone 3.



roadmap · 4 milestones, a measured gate after each

Each diamond is the gate that closes a milestone; the checklists below are the full definition of done behind it. No dates are set: the order and the gates are the commitment, and each milestone is tagged when its gate is met.

Milestone 1 — single target (tag v0.1)

- ☐ CMake project builds `libradar`, `radar_sim`, and the tests with one command on a clean machine (FetchContent, no manual installs)
- ☐ CV truth model, Gaussian sensor noise, 4-state Kalman filter with Joseph-form update
- ☐ `radar_sim --config scenarios/single.json --out out/ --track` writes truth.csv, meas.csv, tracks.csv
- ☐ Console prints raw RMSE and track RMSE; track RMSE is at most 0.5 × raw RMSE
- ☐ `tools/plot.py` produces a figure (truth line, measurement dots, track line); the figure is embedded in the README
- ☐ test_kalman.cpp passes (convergence, symmetry); GitHub Actions builds and runs the tests on every push

Milestone 2 — multiple targets with clutter (tag v0.2)

- ☐ P_D < 1, Poisson clutter, shuffled measurements, hidden origin tags

- ☐ Gating (χ^2 99%), greedy nearest-neighbor association, tentative/confirmed/coasting/deleted lifecycle (3 of 5; delete after 5 misses)
- ☐ `radar_score` computes RMSE, continuity, false tracks, confirmation delay, and mean NIS into metrics.json
- ☐ baseline.json clears every acceptance threshold; crossing.json keeps both identities; dense_clutter.json has at most 1 false track
- ☐ Plot shows track ids and 95% error ellipses; the README documents the σ_a tuning result (mean NIS before and after)
- ☐ clang-tidy, clang-format, and ASan/UBSan jobs added to CI and green

Milestone 3 — two services over UDP (tag v0.3)

- ☐ `net/codec`: binary framing per Data formats, CRC-32, multi-part scans; round-trip and corruption tests pass
- ☐ `radar_sim --udp` streams scans in real time (`--realtime`) or as fast as possible
- ☐ `radar_tracker`: receiver thread → bounded queue → tracker thread → CSV writer and UDP publisher; clean shutdown on SIGINT
- ☐ ThreadSanitizer run is clean on `net` and `tracker` tests
- ☐ `docker compose up` runs simulator and tracker as two services and leaves metrics.json in the mounted volume
- ☐ UDP-path tracks.csv matches the in-process tracks.csv on the same seed (zero dropped datagrams on localhost)

Milestone 4 — live display, optional (tag v0.4)

- ☐ `radar_view` subscribes to TRACKS datagrams and draws a PPI-style scope: detections, tracks with ids, 20-scan trails, error ellipses
- ☐ 60 fps on a laptop with 10 tracks and $\lambda = 20$ clutter
- ☐ Pause and replay from a recorded datagram log
- ☐ Stretch: a σ_a slider that sends a retune message to the tracker (new msg_type)
- ☐ 15-second demo GIF at the top of the README

Risks, open questions, and extensions

The hard parts are filter tuning and association under clutter, not the C++; the plan guards both with measured checks and keeps the deferred decisions explicit.

Risks

Risk	Effect	Mitigation
Kalman mis-tuning (σ_a against σ_m)	tracks lag in turns or chase noise	start from the defaults in this spec; use mean NIS as the tuning instrument; maneuver.json runs in CI
Track swaps when targets cross	continuity fails on crossing.json	Hungarian assignment (minimum total d^2) in place of

Risk	Effect	Mitigation
		greedy; document residual swaps; JPDA as an extension
C++ learning curve (Eigen templates, CMake)	slow Milestone 1	keep M1 tiny; copy Eigen fixed-size idioms from the references; write the CMake presets once
FetchContent build time (Eigen, GoogleTest, json)	slow CI and local rebuilds	cache `build/_deps` in CI; a Docker layer for dependencies; optional system packages
Floating-point differences across compilers	thresholds pass on one platform, fail on another	thresholds with margins instead of byte comparison; same seed; `-ffp-contract=off` in the ci preset
UDP datagram loss or reordering	missing scans, misordered updates	sequence numbers, per-scan reassembly, drop-and-log policy; localhost testing shows zero loss
Scope creep toward the GUI	M2 metrics never finished	M4 is gated behind M3's definition of done; the README plot is the demo until then
Overstating standards compliance	reviewer pushback	README says "aligned with JSF++ and MISRA practice", never "compliant", and lists the specific rules followed

Open questions (decide by the end of the milestone named)

- ☐ Measurements: stay Cartesian with the linear filter, or switch to range and bearing with an EKF or converted measurements? End of M2.
- ☐ Association: is greedy nearest neighbor enough on crossing.json, or does M2 need the Hungarian algorithm? End of M2.
- ☐ Track initiation: single-point with σ_v0 = 150 m/s, or two-point differencing? After the M1 plots.
- ☐ Live view: a Dear ImGui desktop app, or a WebSocket bridge to a browser view that reuses the author's web stack? Before M4.
- ☐ License: MIT or Apache-2.0. Before the repository goes public.
- ☐ Config format: JSON for v1; revisit TOML only if comments in scenario files become necessary.

Extensions (after v1, in rough order of payoff)

- Interacting Multiple Model (IMM) filter: CV and CT models blended by likelihood; removes the lag-versus-noise trade-off in turns.

- Hungarian assignment, then JPDA for dense clutter.
- Range and bearing measurements with an EKF or UKF, including range-dependent noise.
- 3D state (altitude, climb rate) and a second sensor with a registration bias to estimate (sensor fusion).
- Record and replay: log datagrams to a file and replay at any speed, for the live view and for debugging.
- Real data: feed ADS-B aircraft positions (dump1090 with an RTL-SDR dongle) in as measurements and track live traffic.
- Benchmark mode: time per scan against target count and clutter rate, plotted in the README.
- ROS 2 node wrappers so the tracker drops into a robotics stack.

References and learning resources

Start with the first two; they cover every equation in this spec. The rest are for the multi-target parts and the tooling.

- Labbe, R. Kalman and Bayesian Filters in Python. Free Jupyter book; the chapters on the multivariate Kalman filter, process-noise design, and filter consistency map directly onto Milestones 1 and 2.
- Welch, G. and Bishop, G. An Introduction to the Kalman Filter. UNC-Chapel Hill TR 95-041; a short derivation of the predict and update equations used here.
- Bar-Shalom, Y., Li, X. R., Kirubarajan, T. *Estimation with Applications to Tracking and Navigation* (Wiley, 2001). The standard text: gating, M-of-N logic, NIS consistency tests.
- Blackman, S. and Popoli, R. *Design and Analysis of Modern Tracking Systems* (Artech House, 1999). Multi-target tracking, association, track management.
- Kalman, R. E. (1960). A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering*, 82(1), 35–45. The original paper.
- Stone Soup. Open-source Python tracking framework from Dstl; useful for cross-checking results on the same scenario.
- Eigen documentation, in particular fixed-size matrices and the LLT/LDLT solvers for S^{-1}.
- GoogleTest primer.
- nlohmann/json and CLI11.
- Dear ImGui and ImPlot, for Milestone 4.
- CMake FetchContent and GitHub Actions workflow syntax.
- Joint Strike Fighter Air Vehicle C++ Coding Standards (Lockheed Martin, 2005) and MISRA C++, the two standards the coding conventions are aligned with.